MESTRADO EM CIÊNCIA DA COMPUTAÇÃO / UFMA

Entendendo o projeto desde o início

Como modelos de inteligência artificial foram avaliados para detectar vulnerabilidades em código, quanto custaram e o que os resultados ensinam.

UM GUIA PARA QUEM ESTÁ COMEÇANDO DO ZERO

Conceitos explicados antes das siglas

Dataset, experimentos, resultados e decisões

Exemplos didáticos, gráficos, tabelas e glossário

Base documental consultada em 9 de outubro de 2026

Relato do trabalho realizado e de seus limites, com próximos passos separados.

Como ler este guia

Este documento foi escrito para uma pessoa que ainda não conhece programação, inteligência artificial ou pesquisa experimental. A leitura começa pelos conceitos, passa pelos dados e chega aos resultados reais do projeto. Não é necessário entender matemática avançada.

A pergunta em uma frase
É possível usar um modelo menor e mais econômico para reconhecer falhas de segurança em código, sem confiar em pontuações que escondem o problema?

Percurso	Páginas	O que você vai entender
1. Começar do zero	3-6	Código, vulnerabilidade, modelos e origem do projeto.
2. Conhecer os dados	7-11	PrimeVul, rótulos, partições, pares e versões.
3. Aprender a avaliar	12-15	Erros, F1, AUC, limiares e avaliação pareada.
4. Acompanhar a execução	16-21	Infraestrutura, prompting, ajuste e pilotos.
5. Interpretar os resultados	22-27	Desempenho, sementes, recursos e auditoria.
6. Situar o estado atual	28-34	Limites, entregas, próximos passos, glossário e fontes.

Três cuidados durante a leitura

Exemplo didático significa uma situação inventada para ensinar. **Resultado do projeto** significa um número extraído dos registros locais. **Planejado** significa que a proposta existe, mas não há evidência de execução naquele bloco.

Os resultados adotam a consolidação de 08/10/2026 e o manuscrito atual. Notas antigas ajudam a contar a história; interpretações antigas mais fortes foram substituídas pela leitura científica mais cuidadosa da versão atual. As fontes [S1]-[S9] e [R1]-[R6] estão nas páginas 33-34.

Primeiro: qual problema queremos resolver?

Um programa é um conjunto de instruções que diz ao computador o que fazer. Essas instruções são o **código-fonte**. Uma **função** é um trecho organizado desse código que realiza uma tarefa: ler um arquivo, copiar dados ou processar uma mensagem, por exemplo.

Uma vulnerabilidade é uma falha que pode comprometer a segurança. Dependendo do caso, ela pode permitir acesso indevido a dados, execução de comandos ou interrupção de um serviço. Nem todo erro de programa é uma vulnerabilidade, e reconhecer uma falha pode exigir contexto que não aparece em uma única função.

Um exemplo sem precisar saber programar

Imagine uma função que copia uma mensagem para um espaço com capacidade de oito posições. Se ela aceita uma mensagem maior sem conferir o limite, pode escrever fora do espaço reservado. Uma correção possível é verificar o tamanho antes da cópia.

Versão vulnerável - exemplo	Versão corrigida - exemplo
Receber mensagem Copiar mensagem para o espaço reservado	Receber mensagem **Conferir se ela cabe** Se couber, copiar; caso contrário, rejeitar

O que o projeto pede ao modelo
Receber o corpo de uma função C/C++ e indicar se ela deve ser classificada como vulnerável. A avaliação compara a resposta com o rótulo do dataset.

O trabalho executado classifica funções. Ele não demonstra localização exata da linha defeituosa nem correção automática de programas. A utilidade possível é apoiar a triagem: ajudar uma pessoa a escolher o que examinar primeiro.

Base: anteprojeto e manuscrito atual [S1, S4]. Exemplo de código criado para ensino.

O que é um modelo de linguagem?

Um modelo de linguagem aprende regularidades a partir de muitos exemplos de texto. Quando é especializado em código, também aprende padrões de programas. Depois do treinamento, usa números internos, chamados **parâmetros** ou **pesos**, para produzir respostas ou representações de uma entrada.

LLM é a sigla em inglês para modelo de linguagem de grande porte. Isso não significa que o modelo seja uma pessoa lendo o código: ele calcula uma saída. Um texto convincente ou uma nota alta não garante que tenha identificado a falha correta.

Termo	Tradução prática no projeto
Qwen2.5-Coder	Família de modelos especializados em código utilizada nos experimentos.
3B e 7B	Aproximadamente 3 e 7 bilhões de parâmetros. B significa bilhão.
Base	Modelo usado como ponto de partida para o ajuste do classificador.
Instruct	Variante preparada para seguir instruções textuais, usada nos testes YES/NO.
Token	Unidade em que o texto é dividido para entrar no modelo; pode ser parte de uma palavra, símbolo ou outro trecho.
Contexto	Quantidade de tokens que o modelo recebe naquela configuração.

Duas maneiras de fazer a mesma pergunta

Por instrução: enviar uma pergunta e interpretar a palavra YES ou NO que o modelo escreve. **Por classificação:** acrescentar uma saída que calcula uma nota para as duas classes. A segunda opção permite variar o limite usado para dar o alarme.

Modelo maior não é garantia
O 7B tem mais parâmetros que o 3B. Isso pode aumentar a capacidade e o custo, mas o desempenho depende da tarefa, dos dados e de como a avaliação é feita.

Base: configurações e fundamentação do manuscrito [S4].

Como o projeto nasceu

O anteprojeto do mestrado em Ciência da Computação do PPGCC UFMA foi organizado em torno desta pergunta: **quão pequeno e barato pode ser um modelo mantendo desempenho na detecção de vulnerabilidades?** A linha indicada nos documentos é Modelagem Computacional.

A proposta combina três assuntos: detectar falhas de segurança, reduzir o custo dos modelos e avaliar com dados e regras mais rigorosos. O trabalho foi concebido como estudo empírico: executar experimentos, comparar resultados e produzir evidências, sem depender da invenção de um novo algoritmo.

Frente inicial	O que se pretendia investigar
1. Referências iniciais	Medir modelos sem especialização adicional para ter um ponto de comparação.
2. Quantização	Usar menos bits e comparar métodos como NF4, AWQ e GPTQ.
3. Ajuste eficiente	Especializar modelos pequenos com QLoRA e saída de classificação.
4. Avaliação e robustez	Contrastar benchmarks e testar mudanças de código que preservam seu significado.
5. Qualidade e custo	Relacionar desempenho, memória, tempo, dinheiro e, no plano amplo, energia.

O dossiê v3 registra um acervo de 50 artigos, após revisar 41 e incorporar nove referências fundamentais. Essa é uma informação do histórico documental. O manuscrito atual de Seminários usa um conjunto menor, de 13 referências científicas verificadas para aquele texto.

Por que o foco ficou mais específico?
Os resultados tornaram importante entender o que as métricas realmente medem. O artigo atual se concentra em Qwen2.5-Coder, PrimeVul, contraste global/pareado e duas implantações do 7B. O plano completo do mestrado é mais amplo.

Base: anteprojeto, dossiê v3 e protocolo [S1, S2].

Linha do tempo do trabalho

Quando	O que os registros mostram
Antes e durante set/2026	Anteprojeto, revisão de literatura, versões do dossiê, protocolo, cronograma e organização das pastas.
10/09/2026	Primeiro commit do repositório disponível: início do registro versionado. Isso não data o começo de todo o estudo.
11-17/09	Montagem dos scripts, primeiros testes por instrução, avaliação pareada, ajustes de contexto e controle do ambiente.
19/09	Decisão de migrar da v0.1 para a release original do PrimeVul; resultados anteriores foram arquivados.
23-25/09	Reexecução das referências na release original e pilotos 3B: treino 1:1 e treino com proporção original.
26-29/09	Investigação do tamanho da função e terceiro piloto 3B, com seleção de benignas de tamanho semelhante.
04-06/10	Treino 7B com três sementes, avaliações NF4/BF16 e teste do terceiro braço 3B.
06-07/10	Correção da pontuação de empates e alinhamento do F1 pelo limiar da validação.
07-08/10	Plano do artigo, consolidação dos registros, estatística, inspeção de pares, redação e ciclos de revisão.
Estado consultado em 09/10	Manuscrito completo, resultados consolidados e PDF SBC local. Há pendências editoriais e extensões experimentais.

O histórico preserva decisões intermediárias e correções. Uma anotação antiga dizendo que algo estava pendente deve ser lida com sua data: os registros posteriores mostram que consolidação, recuperação dos dados e compilação local já avançaram.

Base: histórico Git, notas de execução e registros editoriais [S3-S6]. Datas em horário local quando disponível.

Dataset: o material usado para ensinar e testar

Dataset significa conjunto de dados. Neste projeto, é uma coleção de funções de programas, acompanhadas de informações que permitem identificar cada exemplo e de um rótulo para a tarefa. Pense em um livro de exercícios com gabarito.

O **rótulo** é a resposta usada como referência: **1 = vulnerável; 0 = não vulnerável segundo a rotulação do conjunto**. O zero não é uma prova universal de segurança. A qualidade da pesquisa depende também da qualidade desse gabarito.

Campo real	O que significa
func	O texto do corpo da função que entra no modelo.
target	O rótulo 0 ou 1 usado para comparar com a predição.
project	O projeto de origem, como Chrome ou Linux.
commit_id	Identificação de uma versão ou mudança no histórico do projeto.
idx e big_vul_idx	Identificadores presentes nos arquivos, úteis para rastreabilidade.
cwe	Campo de categoria de fraqueza; pode estar vazio.
hash	Campo fornecido no registro. Não substitui a conferência de identidade e posição realizada na consolidação.

Como esses dados ficam guardados?

Os arquivos usam **JSONL**: cada linha contém um registro estruturado. Os scripts leem o código e o rótulo, e a consolidação usa também posição, identificador, projeto e commit para ligar cada previsão ao exemplo certo.

O rótulo não deve entrar na pergunta
Durante a avaliação, o modelo recebe o código. O rótulo é usado depois, pelo avaliador, para conferir o resultado. Mostrar o gabarito ao modelo tornaria a avaliação inútil.

Base: release original recuperada, esquema JSONL e scripts [S3, S5].

Por que o PrimeVul foi escolhido?

O PrimeVul reúne funções C/C++ de projetos de código aberto. Seu trabalho de construção usa informações de correções de vulnerabilidades e busca melhorar a rotulação, remover duplicações e organizar a separação dos dados no tempo.

Um **commit** é uma mudança registrada no histórico de um projeto. Um **patch** é o conjunto de alterações dessa mudança. Quando o patch corrige uma vulnerabilidade, é possível estudar a função antes e depois da correção. Essa associação exige cuidado: uma mudança pode alterar várias funções, e nem toda função alterada contém o defeito.

O problema de avaliar com exemplos repetidos

Imagine estudar para uma prova e encontrar exatamente a mesma pergunta no teste. Você pode acertar porque reconheceu o exercício, sem conseguir resolver um novo problema. Em modelos, cópias ou exemplos muito semelhantes entre treino e teste podem produzir um efeito parecido: **vazamento de informação**.

Deduplicar
Reduzir cópias ou repetições inadequadas.

>

Separar no tempo
Organizar treino e teste segundo o histórico.

>

Conferir rótulos
Melhorar a relação entre função e falha.

Esses cuidados atendem problemas diferentes. A rotulação busca um gabarito mais correto. Deduplicação e divisão cronológica reduzem algumas formas de vazamento. Nenhum deles, sozinho, garante que o teste seja perfeito.

Uma correção importante da linguagem
Documentos antigos chamavam a avaliação de "descontaminada". A interpretação atual é mais precisa: o PrimeVul reduz riscos conhecidos, mas não prova ausência de contato com esse código no pré-treinamento do Qwen.

A escolha do dataset faz parte do método científico: um teste rigoroso pode revelar limitações que uma avaliação mais fácil não mostraria.

Base: manuscrito e fontes primárias nele verificadas [S4; R1, R2].

Treino, validação e teste

O conjunto principal é dividido em três partes. Elas têm papéis diferentes e não devem ser tratadas como se fossem uma única pilha de exercícios.

Partição	Funções	Vulneráveis	Não vulneráveis
Treino	184.427	5.574	178.853
Validação	25.430	699	24.731
Teste	25.911	695	25.216
Total das três	235.768	6.968	228.800

Treino: é o material usado para atualizar os adaptadores e a cabeça do classificador. É o momento em que o modelo aprende com exemplos e gabaritos.

Validação, ou dev: serve para escolher a época e calibrar limiares de decisão. É como um simulado usado para ajustar a estratégia.

Teste: serve para descrever como a configuração escolhida se comporta em dados separados. O ideal confirmatório é definir as escolhas antes de consultá-lo.

O conjunto é desbalanceado

No teste, apenas **695 de 25.911 funções, ou 2,68%, são vulneráveis**. As demais 97,32% têm rótulo não vulnerável. Isso significa que uma resposta "não" para tudo pareceria acertar muito, embora não encontrasse nenhuma falha.

Como visualizar a proporção
Em cada 1.000 funções com essa proporção, seriam cerca de 27 vulneráveis e 973 não vulneráveis. É uma ilustração arredondada, não outro experimento.

Neste projeto, o teste foi consultado durante o desenvolvimento das receitas. Por isso, o artigo atual chama a análise de **retrospectiva e exploratória**, mesmo que os limiares reportados tenham sido calibrados na validação.

Base: contagens oficiais recuperadas e manuscrito [S4, S5].

O que é a avaliação pareada?

Na avaliação global, cada função é julgada individualmente. Na avaliação **pareada**, a pergunta fica mais exigente: o modelo distingue uma função vulnerável da versão rotulada como sua correção?

Antes
A função com rótulo vulnerável.

>

Depois
A função associada à correção.

>

Comparação
O alarme ou a nota muda na direção esperada?

As versões de um par compartilham muito código e contexto. Isso ajuda a investigar se a resposta acompanha a mudança relevante, em vez de acompanhar apenas o projeto, o estilo ou o tamanho da função. Mas o par precisa ter sido montado corretamente.

Partição pareada	Pares	Funções nos pares
Treino pareado	4.354	8.708
Validação pareada	562	1.124
Teste pareado	564	1.128

564 pares não são 564 funções
Cada par tem dois membros. Por isso, 564 pares correspondem a 1.128 registros. O conjunto pareado é um recorte específico e não uma nova coleção a ser somada automaticamente às contagens do conjunto global.

Também não há um par de teste para cada uma das 695 funções vulneráveis do teste global. São construções relacionadas, mas com denominadores diferentes. Comparar percentuais sem dizer qual conjunto foi usado pode confundir a leitura.

Nos experimentos do projeto, as receitas de treino foram montadas a partir do treino global. Os pares de validação foram usados para escolher checkpoints; os pares de teste foram usados para avaliar as previsões.

Base: partições pareadas e definições do manuscrito [S4, S5].

A mudança de versão do dataset

O projeto começou usando a versão **PrimeVul-v0.1**. Depois, a conferência mostrou que ela era um subconjunto diferente da distribuição original do artigo. A v0.1 adiciona metadados, mas conserva apenas vulnerabilidades para as quais certos metadados foram recuperados.

Comparação	Release original	v0.1
Treino global	184.427 funções	175.797 funções
Vulneráveis do treino	5.574	4.862
Teste global	25.911 funções	24.788 funções
Vulneráveis do teste	695	549
Teste pareado	564 pares	435 pares
Validação pareada	562 pares	480 pares

Em 19/09, a decisão foi usar a **release original** como referência. As rodadas anteriores ficaram preservadas em **resultados/v01/**, e os testes principais foram refeitos na distribuição original. Portanto, seus números não devem ser misturados em uma mesma comparação sem identificação.

O que passou a ser conferido?

Os scripts verificam os seis arquivos esperados e suas contagens por rótulo. Os registros de execução identificam a release. A consolidação posterior recuperou os arquivos oficiais, calculou hashes completos atuais e comparou os prefixos históricos disponíveis.

Hash é uma impressão digital do arquivo
Ajuda a perceber se os bytes mudaram. Um hash conferido hoje não passa a ser um hash completo registrado no passado. E um arquivo idêntico à fonte oficial ainda pode conter rótulos ou correspondências problemáticas.

A lição é simples: antes de comparar modelos, é preciso confirmar que todos responderam à mesma prova.

Base: downloader, README experimental, arquivo v01 e auditoria [S3, S5].

Como contar os acertos e os erros

O avaliador compara duas coisas: o rótulo de referência e o que o detector respondeu. Isso produz quatro situações. "Positivo" significa que o detector apontou uma vulnerabilidade.

O que existe no gabarito	O detector diz vulnerável	O detector diz não vulnerável
Vulnerável	**Verdadeiro positivo (VP)** Encontrou a falha.	**Falso negativo (FN)** Deixou a falha passar.
Não vulnerável	**Falso positivo (FP)** Deu um alarme falso.	**Verdadeiro negativo (VN)** Não gerou alarme indevido.

Um exemplo para usar nas próximas páginas

Considere uma coleção fictícia de 1.000 funções: 30 vulneráveis e 970 não vulneráveis. Um detector acha 12 das vulneráveis e dispara 48 alarmes falsos. Ele deixa passar as outras 18 vulneráveis e acerta 922 não vulneráveis.

VP	FN	FP	VN	Total
12	18	48	922	1.000

A acurácia desse detector seria (12 + 922) / 1.000 = **93,4%**. Parece boa até perceber que um detector dizendo "não vulnerável" para tudo teria **97%** de acurácia nesse exemplo e encontraria zero vulnerabilidades.

Por que acurácia sozinha engana aqui?
A classe não vulnerável domina a coleção. Acertar essa classe pode esconder o fracasso justamente na tarefa mais importante: encontrar as falhas.

Essa é a razão de o projeto combinar métricas. Cada uma responde a uma pergunta diferente sobre utilidade, erros e esforço de revisão.

Base: método do manuscrito [S4]. Exemplo numérico criado para ensino.

Precisão, revocação, F1 e falsos alarmes

Precisão: quando o detector dá um alarme, quantas vezes ele está certo? No exemplo: 12 acertos / 60 alarmes = **20%**. A cada cinco alarmes, só um corresponde a vulnerabilidade no gabarito.

Revocação, ou recall: de todas as vulnerabilidades, quantas foram encontradas? São 12 / 30 = **40%**. O detector encontra quatro de cada dez falhas.

F1: combina precisão e revocação pela média harmônica. Ele cai quando uma das duas é baixa. No exemplo: 2 × 0,20 × 0,40 / (0,20 + 0,40) = **0,267**, ou 26,7%.

FPR: entre as funções não vulneráveis, quantas receberam alarme falso? São 48 / 970 = **4,95%**. O denominador é o total não vulnerável, e não o total de alarmes.

FNR: entre as vulnerabilidades, quantas passaram despercebidas? São 18 / 30 = **60%**. É o complemento da revocação nesse conjunto.

Precisão e FPR não são a mesma coisa
Precisão olha para a caixa de alarmes. FPR olha para todas as funções não vulneráveis. Quando há muitas funções não vulneráveis, mesmo uma FPR aparentemente pequena pode gerar muito trabalho.

Um exemplo com a dimensão real do teste

Há 25.216 funções não vulneráveis no teste global do projeto. Uma FPR de 1% representaria aproximadamente 252 alarmes falsos; uma FPR de 0,5%, cerca de 126. Isso é uma conta ilustrativa sobre o tamanho do conjunto, não a medição de um detector específico.

Nos resultados, F1 = 0,262 significa uma pontuação F1 de 26,2%. Não significa acertar 26,2% de todas as funções. A unidade e a definição importam tanto quanto o número.

Base: definições usadas no manuscrito [S4]. Contas sobre o exemplo fictício da página 12.

Notas, limiares e AUC

Um classificador pode gerar uma **nota**, ou escore, em vez de apenas sim ou não. No projeto, ela é a diferença entre duas saídas numéricas, chamadas logits: saída vulnerável menos saída não vulnerável. Quanto maior a nota, maior a preferência relativa pela classe vulnerável. Ela não é, por si só, uma probabilidade calibrada.

O **limiar** é o ponto a partir do qual o alarme é disparado. A regra usada é **nota > limiar**. Mover o limiar muda quantos alarmes são dados. Um limiar alto costuma reduzir alarmes falsos, mas pode deixar mais falhas passar.

Limiar utilizado	Como foi escolhido	Para que serve
Maior F1	Buscar o maior F1 na validação global.	Calcular F1, precisão, revocação, FPR e categorias pareadas.
Poucos falsos positivos	Limitar FPR a até 0,5% na validação global.	Observar FNR e FPR reais no teste, com a mesma regra.

São **dois limiares diferentes**. Um valor de F1 e uma taxa de falhas perdidas em regime de poucos alarmes podem, portanto, descrever pontos de operação diferentes do mesmo modelo.

AUC: a qualidade da ordenação global

A AUC ROC resume quanto os escores tendem a colocar funções vulneráveis acima de funções não vulneráveis no conjunto global. Uma referência sem discriminação tem AUC 0,5; uma separação perfeita tem 1. Ela não é uma taxa de acerto e não mostra sozinha quantos alarmes serão úteis em um limiar específico.

Cuidado com o nome VD-Score
A medida local de FNR usa um limiar escolhido na validação e aplicado ao teste. O VD-Score oficial escolhe um ponto na própria curva do conjunto avaliado. O artigo atual diferencia os procedimentos e mostra a FPR efetiva do teste.

Base: calibração e métricas do manuscrito [S4].

Duas medidas diferentes dentro dos pares

Uma avaliação pareada pode olhar para as duas **decisões** ou para a **ordem das notas**. Essas medidas têm regras diferentes e não devem receber o mesmo nome.

Categoria	Vulnerável recebe	Corrigida recebe	Leitura
P-C	Vulnerável	Não vulnerável	As duas decisões corretas.
P-V	Vulnerável	Vulnerável	Alarme nas duas versões.
P-B	Não vulnerável	Não vulnerável	Nenhum alarme nas duas.
P-R	Não vulnerável	Vulnerável	Decisões invertidas.

As categorias dependem do limiar. Já a **ordenação pareada** pergunta se a versão vulnerável recebe nota maior que a corrigida. Se sim, o par vale 1; se as notas são iguais, vale 0,5; se a corrigida tem nota maior, vale 0. A média desses créditos é a pontuação reportada.

Exemplo: ordenar certo sem acertar as duas decisões
Notas 0,8 e 0,7 colocam a vulnerável acima da corrigida. Mas, se o limiar for 0,5, as duas recebem alarme: categoria P-V. Ordenação favorável não garante P-C.

Dar meio crédito aos empates mantém a referência de ordenação sem sinal em torno de **50%**, sob simetria entre as versões. Esse valor também pode ocorrer com todas as notas empatadas. Não é uma prova de compreensão ou de sua ausência.

O antigo "25% de acaso" para P-C só vale para palpites binários independentes com 50% de chance de cada resposta. Não é uma referência universal para um detector que quase sempre diz não. E não é a referência da ordenação.

A correção de empates foi incorporada ao código em outubro. A consolidação usa meio crédito e preserva as colunas históricas, sem fingir que a regra sempre foi essa.

Base: definições e revisão metodológica [S4, S5].

A bancada de experimentos

Para rodar os modelos, o projeto usou uma **GPU NVIDIA A40 de 48 GB**, alugada no RunPod. A GPU é um processador adequado a muitas operações numéricas em paralelo. Sua memória, frequentemente chamada de VRAM, precisa comportar pesos e dados temporários do processamento.

Um **pod** é o ambiente de execução alugado. A rotina foi criar a máquina, instalar as versões previstas, baixar dados e modelo, executar, salvar resultados e preservar os artefatos antes de encerrar a sessão.

Peça da bancada	Papel
Python, PyTorch e Transformers	Executar scripts, operações numéricas e modelos.
PEFT e bitsandbytes	Aplicar adaptação eficiente e carregar a base quantizada.
Git e repositório	Registrar mudanças de código e documentos ao longo do tempo.
CSV	Guardar tabelas de resultados e informações das execuções.
Logs e receitas	Guardar previsões, decisões e configurações que explicam cada resultado.
Trava de ambiente	Conferir versões e GPU para reduzir mudanças involuntárias.

Por que controlar o ambiente?

Nos testes históricos da v0.1, trocar A40 por RTX A5000 mudou nove de 870 respostas mesmo com as demais escolhas iguais. Em outro teste, uma nova sessão A40 com o ambiente travado reproduziu 870 de 870 respostas. São observações específicas, que justificam controlar a comparação, sem garantir identidade em qualquer execução futura.

Preço usado nos registros
US$ 0,50 por hora de A40. É um preço histórico utilizado para estimativas do projeto; não é uma consulta ao preço atual nem uma fatura real.

Base: README do código, trava de versões e notas de reprodutibilidade [S3, S6].

Primeiros resultados: perguntar YES ou NO

Primeiro foi testado o caminho mais direto: enviar cada função aos modelos Qwen2.5-Coder-3B-Instruct e 7B-Instruct, sem exemplos, e pedir apenas YES ou NO. Esse uso é chamado **zero-shot prompting**: uma instrução sem exemplos adicionais de treino para aquela avaliação.

As rodadas oficiais na release original usaram FP16, limite registrado de 8.192 tokens para o código e decodificação gulosa, sem sorteio da resposta. Um analisador transformou a primeira decisão reconhecida em 0 ou 1. Não houve resposta indefinida nas quatro avaliações consolidadas.

Modelo	F1	Precisão	Revocação	FPR
3B Instruct	0,119	0,102	0,144	3,49%
7B Instruct	0,062	0,038	0,160	11,00%

No teste pareado	P-C	P-V	P-B	P-R
3B Instruct	16	67	463	18
7B Instruct	14	78	454	18

O 3B encontrou 100 das 695 funções vulneráveis no teste global. O 7B encontrou 111, mas gerou muito mais alarmes falsos: 2.774, contra 879 do 3B. Assim, seu pequeno aumento de revocação veio com uma forte queda de precisão.

O que foi aprendido nessa etapa?
Na instrução utilizada, aumentar o tamanho de 3B para 7B não trouxe melhor F1. Nos pares, ambos acertaram as duas versões em poucos casos e quase sempre disseram não para as duas.

Isso descreve os dois modelos e esse prompt. Não demonstra que todas as instruções, modelos ou formas de raciocínio teriam o mesmo resultado. Como as saídas eram palavras, essas rodadas não entram na ordenação contínua de escores.

Base: quatro avaliações oficiais e tabelas consolidadas [S3-S5].

Depois: especializar o modelo com QLoRA

Perguntar a um modelo pronto é diferente de **ajustá-lo**. No ajuste fino, o sistema vê exemplos do treino e recebe uma indicação de quanto sua resposta está errada. O processo modifica componentes treináveis para melhorar a tarefa.

Treinar todos os bilhões de parâmetros seria caro. A **LoRA** mantém a base congelada e aprende matrizes adicionais pequenas, chamadas adaptadores. A cabeça de classificação também aprende. Uma analogia é manter um livro e acrescentar anotações especializadas, em vez de reescrever todas as páginas.

A **QLoRA** combina essa adaptação com a base armazenada em quatro bits. No projeto, a base usa NF4, dupla quantização e cálculo em BF16. O otimizador utilizado foi AdamW convencional; não foram usados os otimizadores paginados do artigo QLoRA.

Base congelada
Qwen2.5-Coder Base em NF4.

>

Parte treinável
Adaptadores LoRA e cabeça de duas classes.

>

Saída
Uma nota para comparar com os limiares.

Três palavras que aparecem nos registros

Época: uma passagem pelos exemplos de treino daquela receita. Três épocas significam rever aquele material três vezes.

Perda, ou loss: um número que indica o erro segundo a regra de aprendizado. Cair no treino é esperado; não basta para provar melhora em dados separados.

Checkpoint: versão salva do ajuste, por exemplo ao fim de uma época. A validação ajuda a escolher qual será avaliada.

Ajuste eficiente não garante aceleração
Economizar memória durante o treino não garante inferência mais rápida. A velocidade também depende de operações, bibliotecas, GPU, lotes e da forma de implantar o adaptador.

Base: código, receitas e método do manuscrito [S3, S4; R3, R4].

O piloto 3B: três receitas de treino

Antes de gastar mais com o 7B, foram exploradas três formas de montar o material de treino do 3B. Todas mantiveram as 5.574 funções vulneráveis do treino original.

Receita	Não vulneráveis	Épocas	Treino	Custo da rodada
1:1 sorteado	5.574	3	3,37 h	US$ 1,85
Proporção original	178.853	1	10,16 h	US$ 5,14
1:1 + tamanho	5.574	3	4,98 h	US$ 2,65

1:1 sorteado: para cada vulnerável, usar uma não vulnerável sorteada. As classes ficam equilibradas: 11.148 funções por época.

Proporção original: usar as 184.427 funções do treino completo. Há aproximadamente 32 não vulneráveis para cada vulnerável.

1:1 com controle de tamanho: selecionar, sem reposição, uma não vulnerável cujo número de tokens visíveis seja próximo ao da vulnerável. A intenção é reduzir a informação que o comprimento fornece sobre a classe.

Por que não comparar apenas o custo total?

Uma receita vê muito mais exemplos; outras repetem conjuntos menores por três épocas. O custo da tabela inclui o tempo total registrado da rodada, enquanto a coluna de horas indica treino. As receitas não tiveram orçamento de computação igual.

Uma exploração, não um teste causal isolado
As configurações diferem em composição, volume de exemplos e duração. Compará-las mostra padrões úteis, mas não permite atribuir toda diferença a uma única mudança, como balanceamento ou tamanho.

Cada receita 3B teve uma semente. O terceiro braço nasceu depois dos resultados dos dois primeiros, como investigação de uma possível característica superficial aprendida pelo modelo.

Base: receitas oficiais, notas históricas e tabela consolidada [S3-S5].

O achado que mudou a interpretação: tamanho

No conjunto global, funções vulneráveis tendem a ser mais longas que as não vulneráveis. Isso permite uma regra muito simples: atribuir maior nota às funções com mais tokens. Ela não precisa ler o significado do código.

O projeto mediu exatamente essa referência, chamada **heurística de comprimento** ou baseline de tamanho. Seu escore é o número de tokens visíveis após o corte em 2.048 tokens. O limiar de maior F1 foi escolhido na validação, como nos classificadores.

Resultado real da heurística
No teste global: F1 = **0,200** e AUC = **0,810**. Nos 564 pares: ordenação = **21,10%**, dando meio crédito aos empates. A mesma regra parece útil em uma avaliação e desfavorável na outra.

Como isso pode acontecer?

Considere uma função fictícia com 100 tokens e uma falha de limite. A correção acrescenta uma verificação e passa a ter 110 tokens. Uma regra "mais longo = mais vulnerável" dá nota maior à versão corrigida. Ela segue uma correlação do conjunto global, mas erra a direção da mudança relevante.

Exemplo fictício	Antes	Depois
Rótulo de referência	Vulnerável	Corrigida
Comprimento	100 tokens	110 tokens
Nota da heurística	100	110
Efeito	A versão vulnerável fica abaixo	A versão corrigida parece mais suspeita

Isso é **aprendizado por atalho**: usar uma pista correlacionada à resposta em vez da propriedade que se deseja medir. Os resultados são compatíveis com essa preocupação; o estudo não identificou de forma causal tudo o que os modelos aprenderam.

Até uma regra invertida de comprimento pode parecer favorável nos pares. Por isso, ordenar bem também não prova compreensão de segurança.

Base: baseline de comprimento e discussão atual [S4, S5]. Exemplo inventado para ensino.

O controle de tamanho e a chegada ao 7B

O terceiro piloto procurou evitar que a classe pudesse ser reconhecida só pelo comprimento. Para cada vulnerável, foi escolhida uma não vulnerável com tamanho tokenizado próximo, limitado a 2.048 tokens. O número de exemplos permaneceu 1:1.

A ideia é comparável a ensinar alguém a reconhecer frutas sem deixar que todas as maçãs sejam grandes e todas as laranjas pequenas. Ao equilibrar o tamanho, essa pista deixa de resolver a tarefa no treino. Outras pistas ainda podem existir.

Como foi escolhida a época?

Historicamente, a escolha usou a **ordenação estrita** dos 562 pares de validação, contando empates como erro, e a AUC de uma amostra de validação como desempate. A correção posterior deu meio crédito aos empates. Nos quatro treinos com controle de tamanho, foi possível refazer essa verificação: as épocas selecionadas não mudaram.

Nos pilotos 3B 1:1 e proporção original, os empates por época não foram registrados. Os checkpoints históricos foram mantidos. Na proporção original só houve uma época, portanto não existia outra época a escolher.

Configuração com controle de tamanho	Semente	Época selecionada
3B	1	3
7B	1	3
7B	2	2
7B	3	3

Por que repetir com sementes diferentes?
A semente controla sorteios do treinamento, como inicialização e ordem dos exemplos. Treinar com sementes 1, 2 e 3 ajuda a enxergar variação. Não cria três novos conjuntos de teste.

O 7B repetiu a receita 1:1 com controle de tamanho por até três épocas. Cada treino levou cerca de 7,9 horas. Em seguida, o checkpoint selecionado de cada semente foi avaliado nas duas implantações, NF4 e BF16.

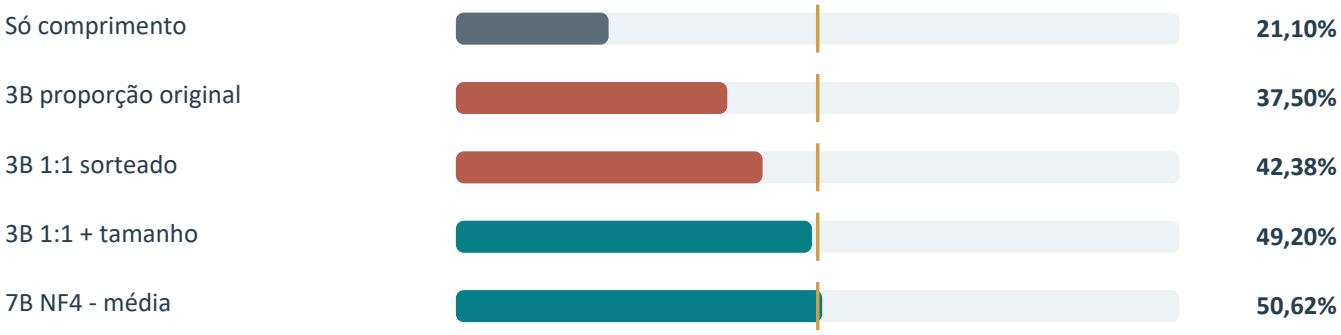
Base: receitas, seleção de épocas e análise atual [S3-S5].

O que os resultados principais mostram?

Configuração	F1 global	AUC global	Ordenação pareada
Só comprimento	0,200	0,810	21,10%
3B 1:1 sorteado	0,195	0,795	42,38%
3B proporção original	0,262	0,851	37,50%
3B 1:1 + tamanho	0,121	0,746	49,20%
7B NF4 - média de 3	0,128	0,743	50,62%

A linha do 7B resume as três sementes e está arredondada. A ordenação usa 564 pares e meio crédito nos empates. Prompting não aparece nesta tabela porque gera decisões, não os escores contínuos usados aqui.

Ordenação pareada: onde a comparação muda



A marca dourada indica 50%, referência de ordenação sem preferência por uma versão. O gráfico descreve estimativas; a incerteza do 7B está na página seguinte.

A principal lição dos experimentos
O melhor F1 global entre essas configurações, 0,262, veio do 3B com proporção original. Mas sua ordenação pareada ficou em 37,50%. Ter a melhor métrica global não significou distinguir melhor a função da sua correção.

O controle de tamanho acompanhou pontuações globais menores e ordenação próxima de 50%. Esse padrão merece investigação, mas as receitas diferentes não permitem concluir que a mudança de tamanho foi a única causa.

Base: tabelas 3 e 4 da consolidação [S5]. Todos os F1 abaixo usam limiar da validação.

Por que uma única semente não basta?

Semente 7B	NF4 - ordenação	BF16 - ordenação
1	55,23%	53,28%
2	45,04%	44,68%
3	51,60%	51,95%
Média	50,62%	49,97%

Olhar só para a semente 1 poderia sugerir um resultado favorável; olhar só para a 2 sugeriria o oposto. O desvio padrão entre as três sementes foi de **5,17 pontos percentuais em NF4** e **4,63 em BF16**. São apenas três treinamentos, portanto a caracterização da variação continua limitada.

O que é o intervalo de incerteza?

A consolidação repetiu a conta em 10.000 reamostragens dos grupos de pares. Esse procedimento é chamado **bootstrap**. A unidade sorteada foi projeto/commit: 550 grupos contêm os 564 pares, e todos os pares de um grupo permanecem juntos.

Medida	Estimativa	Intervalo de 95%
NF4	50,62%	47,92% a 53,33%
BF16	49,97%	47,14% a 52,73%
Diferença NF4 - BF16	0,65 p.p.	-1,22 a 2,54 p.p.

A leitura correta
Os intervalos são condicionais aos três modelos observados. Incluem 50% nas médias e diferenças nos dois sentidos entre implantações. Isso não prova acaso nem equivalência, e não prevê com precisão novos treinamentos.

As três sementes responderam aos mesmos 564 pares: não temos 1.692 pares independentes. O bootstrap manteve o alinhamento entre sementes e implantações. Commits do mesmo projeto ainda podem ser dependentes; nenhum teste de hipótese foi realizado nessa consolidação.

Base: tabela 6, intervalos e comparação alinhada [S5].

Quantização: memória e tempo não andam juntos

Quantizar é armazenar os pesos com uma representação numérica mais compacta. **NF4** é uma representação de quatro bits; **BF16** usa 16 bits. Menos bits pode economizar memória, mas o resultado real depende também da forma como o cálculo é executado.

Cada checkpoint 7B foi avaliado de duas maneiras: **base NF4 com adaptador separado** e **base BF16 com adaptador mesclado**. Mesclar significa incorporar a atualização LoRA aos pesos. As duas configurações mudam quantização e mesclagem ao mesmo tempo.

Recurso	NF4 + adaptador	BF16 mesclado
Memória após carga	4,63 GB	14,15 GB
Pico de memória registrado	12,49 GB	20,76 GB
Tempo por função no teste	59,6-60,0 ms	46,8-47,1 ms
Custo estimado do teste	US$ 0,2145-0,2160	US$ 0,1684-0,1696

Resultado observado nesse ambiente
NF4 usou cerca de **67% menos memória após a carga** e **40% menos no pico**. Porém, o tempo amortizado por função foi cerca de 28% maior. BF16 foi aproximadamente 22% mais rápido em tempo por função.

O teste usou A40, Transformers, contexto de 2.048 tokens e lotes de até 32.768 tokens. O tempo é o total de pontuação do teste dividido pelo número de funções; não é a espera de uma chamada individual. O pico foi medido na pontuação de validação, teste e pares, podendo não incluir a mesclagem.

As diferenças de ordenação não foram distinguíveis nos intervalos reportados. Isso compara duas implantações completas, sem isolar o efeito da quantização. Também não certifica economia de energia, faturamento real ou desempenho em outra GPU.

Base: tabelas de recursos e método [S4, S5].

E se só pudermos aceitar poucos alarmes falsos?

Um detector pode ser difícil de usar se encher a caixa de revisão com alarmes falsos. O projeto também avaliou um ponto mais restritivo: escolher um limiar que permita FPR de até **0,5% na validação** e aplicá-lo ao teste sem reajuste.

Configuração	Vulnerabilidades perdidas no teste (FNR)	Alarmes falsos no teste (FPR)
Só comprimento	100,00%	0,00%
3B 1:1 sorteado	94,10%	0,54%
3B proporção original	88,06%	0,65%
3B 1:1 + tamanho	96,26%	0,63%
7B NF4 semente 1	97,99%	0,69%
7B NF4 semente 2	98,71%	0,38%
7B NF4 semente 3	98,27%	0,56%

Nesse ponto, a configuração 3B com proporção original perdeu aproximadamente 88 de cada 100 vulnerabilidades. O 7B perdeu cerca de 98 de cada 100. A redução de alarmes veio com uma grande perda de sensibilidade.

O limite da validação não é garantia no teste
O limiar fica fixo, mas a distribuição dos escores muda entre conjuntos. Por isso, várias linhas ultrapassaram 0,5% de FPR no teste. O artigo mostra o valor efetivo, em vez de afirmar que a restrição foi atendida em todas as configurações.

No baseline de comprimento, o limiar restritivo coincidiu com o teto de 2.048 tokens. Como a decisão exige nota estritamente maior, nenhuma função disparou alarme: FPR zero e FNR 100%. É uma consequência dessa combinação de corte e regra, não uma incapacidade universal de toda heurística de tamanho.

Base: FNR* e FPR* das tabelas consolidadas [S5].

Consolidar: transformar registros em evidência

Depois das execuções, o projeto tinha tabelas, logs e receitas espalhados no histórico. A consolidação de 08/10 reuniu essas fontes para conferir identidades, refazer métricas e produzir tabelas e análises rastreáveis.

Material coberto	Quantidade	Papel
Avaliações	14	Quatro por instrução, nove de classificadores e uma heurística.
Arquivos de previsão	33	Quatro de prompting, 27 dos classificadores e dois do tamanho.
Receitas de treinamento	6	Três receitas 3B e três sementes 7B já existentes.
Arquivos do dataset	6	Treino, validação e teste, globais e pareados.

Um mesmo classificador produz arquivos para validação, teste e pareado; por isso, número de avaliações e número de logs não são iguais. A conferência alinhou cada previsão à posição, ao identificador, ao projeto, ao commit e ao rótulo da fonte oficial.

Por que a identidade importa?

Se o log de uma função for ligado ao rótulo de outra, a métrica fica errada mesmo que a fórmula esteja certa. Alguns membros corrigidos são reutilizados na release oficial; por isso, juntar apenas pelo identificador poderia multiplicar ou perder registros. A consolidação preservou as posições oficiais.

O que a conferência conseguiu e o que não conseguiu
As métricas principais reproduziram a precisão publicada, sem divergências numéricas registradas. Mas os escores salvos têm cinco casas decimais: não é possível recuperar uma precisão interna que nunca foi gravada.

Foram implementados inventário, reconstrução das métricas, bootstrap e exportação. Oito testes científicos passaram. Essa consolidação foi em CPU, sem novo treinamento, sem inferência dos modelos e sem uso de RunPod.

Base: implementação, manifesto, auditoria e suplemento [S5].

Conferir também os pares e o corte do código

Truncamento é cortar a entrada quando ela ultrapassa o limite. Os classificadores receberam até 2.048 tokens do início da função. Se a correção está depois desse ponto, a diferença relevante pode ficar invisível.

Dos 564 pares de teste, **465 cabem integralmente** nesse limite. Em 32 pares, as duas sequências de entrada ficam idênticas depois do corte. As avaliações 7B tiveram de 32 a 40 empates de nota; score igual sozinho não prova entrada idêntica, pois também há arredondamento e outras possibilidades.

Uma inspeção focalizada de correspondência

Foram examinados 15 pares com diferença de cabeçalho ou assinatura. A consulta aos arquivos do commit e do seu pai confirmou **cinco casos em que o membro rotulado como corrigido é outra função**. Eles pertencem a libjpeg, PHP, rizin, Linux e drogon.

Recorte da análise	Pares	Para que serve
Principal oficial	564	Manter a referência do benchmark.
Sem truncamento	465	Verificar o resultado quando o código cabe integralmente.
Sem as cinco correspondências inadequadas	559	Medir a influência desses casos específicos.
Interseção dos dois controles	460	Aplicar ambos os recortes ao mesmo tempo.

Esses cinco casos explicam o resultado?
Nas seis avaliações 7B, removê-los mudou a ordenação em no máximo 0,22 ponto percentual. O padrão geral permaneceu. Isso não certifica os demais 559 pares nem estima a frequência do problema em todo o PrimeVul.

Os recortes são retrospectivos e mudam a composição da amostra. Não substituem o conjunto principal. Também não foi trocada uma função por outra sem previsão histórica disponível.

Base: auditoria de pares, tokenização e tabelas de sensibilidade [S4, S5].

O que podemos concluir com responsabilidade

Evidência observada	Conclusão que ela sustenta
Comprimento: AUC 0,810 e ordenação 21,10%.	Uma característica superficial pode separar classes globalmente e ordenar os pares na direção desfavorável.
Melhor F1 global: 0,262; ordenação 37,50%.	O melhor resultado global desta comparação não identificou melhor as duas versões da correção.
7B: médias de ordenação próximas de 50% e variação entre sementes.	Uma única execução pode sugerir uma história incompleta; os intervalos não provam acaso nem equivalência.
NF4: menos memória, mais tempo amortizado.	Nesse ambiente, as duas implantações trocaram memória por tempo.

Os limites que acompanham essas conclusões

Desenho: o estudo é exploratório e retrospectivo; houve consulta ao teste durante o desenvolvimento. As receitas não têm o mesmo volume nem o mesmo orçamento. Base/Instruct e contextos também diferem entre etapas.

Amostra: há uma semente por receita 3B e três no 7B; os pares não são completamente independentes. A inspeção semântica dos rótulos não foi completa, e o corte de entrada esconde parte do código.

Generalização: os resultados se referem a uma família de modelos, um dataset, uma GPU e uma biblioteca. Não permitem afirmar que todo LLM falha, que todo ajuste é inútil ou que quatro bits sempre mantêm a qualidade.

O valor científico do trabalho
Ele torna visível a diferença entre obter bons indicadores em uma coleção e discriminar uma falha da sua correção. A contribuição atual é uma caracterização documentada, com medidas de recursos e limites explícitos.

Base: discussão e limitações do manuscrito atual [S4].

O que está pronto e o que continua aberto

Estado	Bloco do projeto
Realizado	Anteprojeto, dossiês, protocolo, cronograma e organização do repositório.
Realizado	Referências por prompting 3B/7B na release original; heurística de comprimento.
Realizado	Três receitas 3B e treino 7B com três sementes; avaliações NF4/BF16 do 7B.
Realizado	Recuperação da release, conferência das previsões, métricas, bootstrap e sensibilidades.
Realizado	Manuscrito completo de Seminários, revisões editoriais, bibliografia e suplemento.
Visualização disponível	Compilação local do manuscrito SBC em PDF de 15 páginas, com inspeção registrada.
Pendente para entrega	Autoria e afiliação completas, regras de extensão e ajuste do comprimento dos resumos.
Sem execução demonstrada neste bloco	Comparação AWQ/GPTQ/INT8, outras famílias e UniXcoder; energia, motor vLLM, robustez por transformações e contraste controlado com benchmarks legados.

A prévia interna de LaTeX do Codex permaneceu com falha de ambiente, mas uma compilação local posterior com MiKTeX gerou o PDF. Os fechamentos anteriores preservam o estado antigo; o README atual registra a atualização.

Revisão do texto não é conclusão do mestrado
O artigo teve ciclos de redação e conferência com Codex e Claude, com registros preservados. Aprovação editorial não é defesa acadêmica, submissão aceita ou validação independente de todos os dados. A apresentação e a submissão não constam como realizadas.

Os planos do artigo indicam 11/11/2026 como data-alvo de Seminários. Isso é uma data planejada nos documentos, não confirmação das regras atuais da disciplina.

Base: README atual, consolidação e registros de fechamento [S5, S6].

Como continuar a pesquisa a partir daqui

Os próximos passos têm dois níveis: terminar a entrega do artigo atual e desenhar novas evidências para o projeto mais amplo. Eles não precisam ser confundidos.

Para terminar o artigo de Seminários

Fechar identificação e formato: completar autoria e afiliação, confirmar limite de páginas e ajustar resumos e apresentação às regras da disciplina ou do veículo.

Preservar a rastreabilidade: manter tabelas, fontes, suplemento, receitas, scripts e versões ligados ao manuscrito final. Uma nova revisão deve identificar qual versão dos resultados utiliza.

Para fortalecer a evidência experimental

Definir uma avaliação confirmatória: fixar receitas, critérios e limiares antes de consultar um novo teste. Isso reduz a influência das observações usadas para desenvolver as escolhas.

Ampliar repetições e comparabilidade: usar mais sementes e controlar o orçamento de treino quando a pergunta for sobre composição do dataset ou escala do modelo.

Revisar correspondências e contexto: examinar de forma sistemática os pares e planejar como tratar código que não cabe no limite de entrada.

Separar efeitos de implantação: por exemplo, incluir BF16 com adaptador separado para distinguir mesclagem de quantização; repetir medições de recursos e latência individual.

> **Para retomar o plano amplo do mestrado**
> AWQ, GPTQ, INT8, outras famílias, UniXcoder, energia e robustez continuam como possibilidades do planejamento. Sua prioridade e viabilidade devem ser ajustadas à pergunta de pesquisa, à orientação e aos recursos disponíveis.

A decisão mais útil agora é escolher qual pergunta precisa de nova evidência. Acrescentar mais experimentos sem uma pergunta clara pode aumentar o custo sem melhorar a conclusão.

Base: conclusão do manuscrito e distinção entre recorte atual e plano amplo [S2, S4, S6].

Glossário para consultar durante a leitura

Termo	Significado neste guia
Ajuste fino	Treinamento adicional para adaptar um modelo a uma tarefa.
Adaptador LoRA	Matrizes pequenas treináveis acrescentadas a uma base congelada.
AUC ROC	Resumo da discriminação global dos escores ao variar limiares. Não é acurácia.
Baseline	Referência de comparação: um modelo inicial ou uma regra simples.
Benchmark	Dados e regras usados para comparar métodos de avaliação.
BF16 / FP16	Duas representações numéricas diferentes de 16 bits.
Bootstrap	Recalcular uma estatística em amostras sorteadas a partir dos registros observados.
Cabeça de classificação	Saída treinável que transforma a representação do código em notas para classes.
Checkpoint	Estado salvo de um modelo ou adaptador em determinado momento.
Commit / patch	Uma mudança registrada / as alterações dessa mudança no código.
Dataset / split	Conjunto de dados / uma partição, como treino ou teste.
Época / loss	Uma passagem pelo treino / a medida de erro usada no aprendizado.
F1 / precisão / recall	Equilíbrio entre precisão e recall / acerto entre alarmes / falhas encontradas.
FPR / FNR	Alarmes falsos entre não vulneráveis / falhas perdidas entre vulneráveis.
Inferência	Usar o modelo para produzir uma saída, sem atualizar seus parâmetros.
Limiar / score	Limite da decisão / nota que será comparada a esse limite.
NF4 / quantização	Formato de quatro bits / representação numérica mais compacta.
Seed / semente	Número que controla os sorteios de uma execução de treinamento.
Token / truncamento	Unidade da entrada / corte da entrada que excede o limite.
VRAM / pico	Memória da GPU / maior alocação observada na janela de medição.

Onde encontrar cada parte no projeto

Pasta	O que ela guarda
01_planejamento/	Anteprojeto, dossiês, protocolo e planos do artigo.
02_referencias/	Artigos, extrações de texto, inventários e conferências de fontes.
03_experimentos/codigo/	Scripts históricos e o bloco novo consolidacao_artigo/.
03_experimentos/dados/	Dataset e materiais recuperados; artefatos grandes ficam fora do Git.
03_experimentos/resultados/	CSVs históricos, arquivo v01/ e resultados derivados do artigo.
03_experimentos/logs/	Previsões e registros das execuções, incluindo material descartado identificado.
04_escrita/artigo/seminarios_2026/	Texto atual, bibliografia, LaTeX, suplemento e histórico editorial.
05_reunioes/	Materiais de reunião e consultas registradas sobre escrita e decisões.

O papel dos scripts, em ordem lógica

00 e 02: baixar/conferir e inspecionar os dados. **01 e ambiente.py:** preparar e conferir o ambiente. **03 e 04:** gerar decisões por instrução e contar categorias pareadas. **05:** comparar logs. **06:** treinar adaptadores. **07:** avaliar classificadores. **08:** medir a heurística de comprimento.

No diretório **consolidacao_artigo/**, os módulos 01-04 inventariam, reconstroem métricas, calculam estatística e exportam tabelas/figura. Eles trabalham sobre registros existentes, sem precisar repetir inferência ou treinamento.

Quando houver versões diferentes
Para os resultados atuais, usar a pasta consolidacao_2026-10-08 e o manuscrito artigo_completo.md. Dossiês, planos, notas e revisões antigos continuam úteis como história, mas podem descrever um estado anterior.

Base: organização atual das pastas e README do código [S3, S6]. Caminhos relativos à raiz do projeto.

Fontes deste relato

A explicação prioriza resultados consolidados e o manuscrito atual. Os caminhos abaixo são relativos a **D:/UFMA UNIVERSIDADE/MESTRADO - ESTUDO LLM/**. Referências [S1]-[S9] identificam documentos do próprio projeto.

[S1] Origem do projeto. 01_planejamento/Projeto_Mestrado_Deteccao_Vulnerabilidade_Eficiente (1).pdf
Anteprojeto, objetivos e escopo amplo.

[S2] Planejamento revisado. 01_planejamento/RECORTE_A_dossie_completo_v3.md; RECORTE_A_protocolo_experimental.md
Evolução das perguntas, literatura e matriz planejada.

[S3] Execução histórica. 03_experimentos/codigo/README.md; 00_baixar_primevul.py; requisitos_travados.txt; resultados/NOTAS_etapa1.md; NOTAS_etapa3.md
Scripts, versões, receitas, resultados e decisões datadas.

[S4] Síntese científica atual. 04_escrita/artigo/seminarios_2026/texto/artigo_completo.md
Método, resultados, interpretação, limitações e referências.

[S5] Evidência consolidada. 03_experimentos/resultados/artigo_seminarios_2026/consolidacao_2026-10-08/
Tabelas, métricas, manifesto, auditoria, intervalos e sensibilidades. Implementação em codigo/consolidacao_artigo/IMPLEMENTACAO_2026-10-08.md.

[S6] Estado editorial atual. 04_escrita/artigo/seminarios_2026/README.md; revisao/reescrita_opus_extra_2026-10-08/FECHAMENTO.md
Texto, revisões, PDF local e pendências. O README inclui atualizações posteriores ao fechamento.

[S7] Suplemento atual.
04_escrita/artigo/seminarios_2026/revisao/reescrita_opus_extra_2026-10-08/07_previa_v003/SUPLEMENTO.md
Detalhes metodológicos e fontes documentais de inspeção.

[S8] Dados recuperados. 03_experimentos/dados/artigo_seminarios_2026/release_original/
Seis JSONL usados para conferir esquema, contagens e identidade.

[S9] Histórico versionado. Histórico Git local, desde o commit db3b15a de 10/09/2026
Datas e sequência das alterações rastreadas; parte do trabalho recente ainda está fora dos commits.

Este guia foi elaborado a partir dos arquivos disponíveis na pasta compartilhada, sem novos experimentos com modelos.

Para lembrar e para aprofundar

O projeto começou com uma pergunta de eficiência. Montou uma bancada, conferiu dados e modelos, corrigiu a escolha da release e mediu uma referência por instrução. Depois, especializou classificadores, investigou o tamanho da função e ampliou o estudo para três sementes do 7B.

A consolidação mostrou por que a avaliação precisa de várias lentes. A coleção global pode premiar uma característica superficial, enquanto o par vulnerável/corrigida revela outro comportamento. A implantação compacta também pode poupar memória e consumir mais tempo. As conclusões atuais descrevem essas configurações, com seus limites.

Você entendeu o projeto se conseguir explicar...
Por que dizer "não" para tudo parece acertar muito; por que 564 pares são 1.128 registros; por que AUC não é acurácia; por que ordenar as versões não garante duas decisões corretas; e por que quatro bits não significam automaticamente mais velocidade.

Seis leituras para aprofundar

[R1] Ding et al. (2024). Vulnerability Detection with Code Language Models: How Far Are We?. Origem do PrimeVul e avaliação.

[R2] Risse e Böhme (2024). Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection. Limites dos detectores e pares.

[R3] Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. Adaptação com poucos parâmetros.

[R4] Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. Ajuste sobre base quantizada.

[R5] Hui et al. (2024). Qwen2.5-Coder Technical Report. Família de modelos utilizada.

[R6] Huang et al. (2026). Do Fine-Tuned LLMs Understand Vulnerabilities? An Investigation into the Semantic Trap. Contrastes de avaliação em código.

Os links remetem às obras identificadas nos registros locais. Este guia não realiza uma revisão sistemática nova nem certifica novidade científica. Seu objetivo é ensinar o caminho percorrido pelo projeto e permitir que os resultados sejam lidos com clareza.

Referências científicas abaixo constam das fontes verificadas no manuscrito atual [S4].